

Week 2 Lesson Plan

Enterprise RAG: The Shape of a Decision

ASIF QAMAR

pepal leaves publisher

www.supportvectors.ai

All rights reserved. The contents in this document are the intellectual property of SupportVectors. No part of it can be shared at all without the explicit permission of the author.

First draft, June 12, 2026

The Shape of a Decision

The true logic of this world is
in the calculus of probabilities.

James Clerk Maxwell

‘Well! I’ve often seen a cat
without a grin,’ thought Alice;
‘but a grin without a cat! It’s
the most curious thing I ever
saw in my life!’

Lewis Carroll, *Alice’s
Adventures in Wonderland*

A Gentle Re-Entry

LAST SATURDAY WE MADE A PROMISE and only half-kept it. We spent the day arguing that meaning had become geometry — that a modern machine does not reason in words but in *vectors*, points on the surface of a high-dimensional globe, where nearness is kinship and direction is meaning. We watched a machine fetch a photograph of a dog when asked, in words, about *learning to love an animal*. We met the curse of dimensionality and its merciful twin, the concentration of measure. We saw that raw embeddings huddle in an anisotropic cone, and we named — only named — the cure: contrastive learning. And several times, when a sharp question arose about *why* the machine lives on a globe, or *how* it learned the geometry in the first place, I asked you to hold the thought. Today the thought is paid back with interest.

HERE IS THE SINGLE THREAD of today. Week 1 *asserted* the geometry; Week 2 *earns* it. We will build, by hand and almost from nothing, the two small machines on which everything downstream rests. The first is

This lesson plan is also your refresher. Read it first for the shape of the argument; return to the book chapters for the full derivations. When time is short before a later week, come back *here*. The plan is dense by design — it is built to repay a second reading, and a third.

the **softmax**: the device that takes a fistful of raw, opinionated numbers and presses them into a *decision* — a proper probability distribution that sums to one. The second is the **negative log-likelihood**: the device that scores a decision by how *surprised* the world leaves it. These two are a matched pair, lock and key. The softmax proposes a belief; the negative log-likelihood grades it; and the grading, pushed downhill by gradient descent, is exactly what bends the geometry into the shape we admired last week.

AND THERE IS A CHARACTER I want you to start watching for, from the very first hour. The softmax is the *Cheshire Cat* of machine learning. It will appear this morning as the way we decide whether a picture is a cow or a duck. It will vanish, and reappear at noon as *attention* — the mechanism by which a word looks up its own meaning in the dictionary of its sentence. It will fade again, and grin at us in the evening from the physics of a coffee cup, from the $\sqrt{d}$ buried in the attention equation, and from a sixty-year-old model of human memory. Each time it returns it will be wearing a different costume, and each time I will ask you the same question: *have we met before?* Keep a corner of your mind on the cat. By nightfall you will see that half of modern machine learning is one idea in many hats.

WHY? If you leave today with one conviction, let it be this: a learned representation is not magic and not a lookup table — it is the fixed point of a *scoring game*. Something proposes a belief (softmax); something measures its surprise (negative log-likelihood); and gradient descent walks the belief downhill until the surprise can fall no further. Every embedding you will ever index in Qdrant, every attention head in every transformer you will ever debug, is a frozen move in that game. Understand the game, and the rest of the course is bookkeeping.

What Must You Carry Forward

WHEN TODAY'S SESSION CONCLUDES, you should be able to articulate — without hand-waving — each of the following.

- Write the softmax equation from memory, and explain in one breath what the exponential does (makes everything positive and amplifies gaps) and what the denominator does (restores the conservation law, so the mass sums to one).
- Explain the role of *temperature*: show what happens to the distribution as $T \rightarrow 0$ (it snaps onto the argmax, a hard decision) and as $T \rightarrow \infty$ (it relaxes to the uniform distribution, a shrug), and connect both limits to the same exponential.
- Motivate surprise as $-\log p$ from first principles: why the inverse $1/p$ captures the intuition but starts at the wrong place, and why the logarithm is the natural repair that sends certainty to zero surprise.
- Give the *rigorous* reason the logarithm is forced, not chosen: that the likelihood is a product, that maximising a product of a thousand

probabilities underflows any floating-point number ever built, and that log is the unique (up to scale) continuous map turning products into sums — so additivity and computability are the same fact, and they hand us the negative log-likelihood.

- Explain attention as an *approximate, differentiable dictionary lookup*: given a query that matches no key exactly, distribute a unit of attention over the nearby keys by proximity, and read off a blended value.
- State the scaled dot-product attention equation, identify which part is the softmax, which is the proximity score, and which is the retrieved meaning, and explain why the worked example must be a polysemous word like *bank*.
- Explain why machines must speak “machinese”: that because they learn by gradient descent, meaning must live on a smooth, continuous manifold where infinitesimal moves are legal — something the discrete lexicon can never offer.
- Describe the bridge from per-token contextual meanings to a single chunk embedding: that attention yields one vector *per word*, and that *pooling* (mean-pooling, usually) collapses them into the one vector we actually index.
- Explain masked language modelling as the “guess the missing word” game, and argue why a machine that consistently wins it must have built genuine contextual understanding — and why its score is exactly the negative log-likelihood you built this morning.
- Explain anisotropy and the cure: that attraction alone collapses a space into a cone, that contrastive learning adds the missing *repulsive* force, and how to *measure* the improvement with within-class versus between-class cosine histograms.
- Read search as motion on a curved surface — the information-geometry view — and explain why a single shared space for text and images (CLIP) is what lets a sentence retrieve a photograph.

If you can do these without flinching, you are ready for Week 3, where we stop admiring the geometry and start *cutting documents* to exploit it.

Act I: The Shape of a Decision

EVERYTHING IN THIS ACT IS ABOUT ONE TRANSFORMATION: how a machine turns raw enthusiasm into a lawful belief, and how we then grade that belief. We will meet the softmax as a sculptor of decisions, and the negative log-likelihood as the honest judge that scores them. By the end of the Act you will have built both with your own hands, and you will understand why they are not two ideas but one idea seen from two sides.

Softmax, Built by Hand

LET US START WITH A FISTFUL OF NUMBERS, and a piece of graph paper. Suppose a small vision model has looked at a photograph and emitted three raw scores — call them *logits* — for the three things it has been taught to recognise: a cow, a duck, a sofa. Say the numbers are 2.0, 1.0, and 0.1. Draw them as three bars. The cow bar is tallest, the duck shorter, the sofa barely off the ground. These numbers are honest reports of the model's enthusiasm, but they are useless as a *decision*. They are not probabilities: they do not sum to anything in particular, one of them could have been negative, and there is no sense in which the cow bar is "2.0 units of belief". We want to turn this raw enthusiasm into something we could bet on — a distribution that sums to one, where each height is the machine's honest probability that the photograph is a cow, a duck, or a sofa.

A *logit* is a pre-belief: a real number, possibly negative, with no obligation to sum to anything. The whole job of the softmax is to civilise a vector of logits into a vector of probabilities.

The first move is to *exponentiate*. Replace each logit z_i by e^{z_i} . Three things happen at once, and all three are the point. First, every exponential is positive, so a negative logit can no longer embarrass us — e^{-3} is small but perfectly legal. Second, the exponential is monotonic, so it never reorders the bars: the cow was tallest and stays tallest. Third — and this is the one to feel in your bones — the exponential *amplifies gaps*. A logit lead of 1.0 becomes a multiplicative lead of $e \approx 2.718$. The rich get richer. Re-draw your bars after exponentiating: $e^{2.0} \approx 7.39$, $e^{1.0} \approx 2.72$, $e^{0.1} \approx 1.11$. The cow now towers.

Now the second move: *normalise*. Add the three exponentials together — here, $7.39 + 2.72 + 1.11 = 11.22$ — and divide each by the total. The bars become 0.659, 0.242, 0.099. They sum to one. We have a distribution. Written out, this two-step ritual is the softmax:

$$\text{softmax}(z)_i = \frac{e^{z_i}}{\sum_j e^{z_j}}.$$

Read the equation aloud as the two gestures you just performed: *exponentiate to make positive and to amplify; divide by the total to restore the*

conservation law. The numerator is enthusiasm; the denominator is the constraint that all enthusiasm must, in the end, share a single unit of probability mass between them.

WHERE DID THE LOGITS COME FROM in the first place? This is worth a moment, because it is the hinge between today's two halves and last week's geometry. In a real classifier, the photograph has already been turned into an *embedding* — a vector, a point on the globe — and each candidate class (cow, duck, sofa) owns its own vector too. The logit for “cow” is nothing more exotic than the *dot product* of the image's embedding with the cow class vector: a measure of how aligned the picture is with the idea of a cow. High alignment, high logit; orthogonal, low. So the softmax does not act on numbers from nowhere — it acts on *geometric alignments*, turning “how close is this picture to each concept” into “how probable is each concept.” Last week's nearness becomes this week's belief through exactly one exponential and one division. Hold that bridge; we will cross it again, in the other direction, when attention scores turn out to be dot products too.

So, the question I leave you with before we turn the temperature knob: this vector [0.659, 0.242, 0.099] sums to one and assigns most of its mass to “cow”. Does it not look exactly like the thing you would want, if you had to *decide* how likely the photograph is a cow rather than a duck? Hold that “looks like a probability” feeling. In a few pages we will ask the harder question — not how to *make* such a belief, but how to *grade* it — and the two halves will snap together.

The Temperature Knob

THERE IS A SINGLE KNOB hidden in the softmax, and learning to turn it is worth an hour of anyone's life. Insert a positive number T , the *temperature*, and divide every logit by it before exponentiating:

$$\text{softmax}_T(z)_i = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}}.$$

At $T = 1$ we recover the plain softmax we just built. The interesting education is in the two extremes, and I want you to compute them by hand for our cow-duck-sofa logits [2.0, 1.0, 0.1], so the limits are not a claim you take on faith but a thing you have seen with your own pencil. Take T small, say $T = 0.1$. Now the logits, divided by T , become [20, 10, 1], and the exponential gaps are astronomical: e^{20} utterly dwarfs e^{10} . Normalise, and the distribution is essentially [1, 0, 0]. The mass

An old physicist's habit: whenever you see a quantity divided by the sum of all such quantities, suspect a *conservation law* hiding underneath. Probability mass behaves like an incompressible fluid — there is exactly one unit of it, and the softmax is merely deciding how to apportion that one unit among the candidates. Push more onto the cow and less must remain for the duck; nothing is created, nothing destroyed, only redistributed. That denominator is the bookkeeper enforcing the conservation, and it is the same instinct by which we account for everything that flows out of a closed surface. Keep your eye on what is conserved, and these equations stop being arbitrary. The name is a small slander. The softmax is not a soft *max* — it does not return the maximum. It returns a *soft argmax*: a smeared-out, polite pointing-at the largest entry that, crucially, is differentiable. The hard argmax has a gradient of zero almost everywhere and is useless to gradient descent; the softmax is its smooth, learnable cousin.

The word *temperature* is not a metaphor borrowed loosely. Tonight, in the rabbit hole, we will derive the softmax from the physics of heat, and T will turn out to be *literally* a temperature — the same T that decides whether a water molecule clings to your coffee or escapes as steam. Watch the cat.

This is exactly the “creativity” slider you have turned in a chatbot’s settings. A low temperature makes the model repeat the single most likely next token, dependable and dull; a high temperature lets long-shot tokens through, surprising and sometimes unhinged. Same knob, same exponential, same conservation law.

has *snapped* onto the single largest logit. This is a *hard*, confident, almost deterministic decision: low temperature is a cold, decisive machine that always bets on its favourite and never hedges. Take T large, say $T = 100$. Now the logits become $[0.02, 0.01, 0.001]$, all nearly equal; the exponentials are all nearly 1; and the distribution relaxes toward the uniform $[\frac{1}{3}, \frac{1}{3}, \frac{1}{3}]$. High temperature is a hot, agitated machine that shrugs — it has opinions but refuses to commit to them.

So temperature controls the *confidence* of a decision without touching the underlying logits. It does not change *what* the machine prefers — the ordering of the bars is untouched, because dividing by a positive constant cannot reorder them — it changes only *how hard* the machine presses on its preference. This single idea, that one can dial a distribution between a snap and a shrug while leaving its preferences intact, will return this afternoon in a place you would never expect: it is, secretly, the reason the attention equation has a $\sqrt{d}$ in its denominator. But I am getting ahead of the cat.

Put the whole sweep in one table, for our cow–duck–sofa logits $[2.0, 1.0, 0.1]$, so you can see the mass migrate as the knob turns:

T	cow	duck	sofa	character
0.25	0.978	0.018	0.004	nearly a hard argmax
0.5	0.866	0.117	0.017	confident
1.0	0.659	0.242	0.099	the plain softmax
2.0	0.484	0.293	0.223	hedging
10	0.371	0.336	0.293	almost a shrug

Trace the cow column down: at low temperature it owns the mass; as the temperature climbs it surrenders, share by share, to the duck and the sofa, until at $T = 10$ the three are nearly equal. At no point does the ordering flip — the cow leads throughout — yet the *confidence* melts from near-certainty to a near-shrug. That is temperature, entire.

The Trick That Keeps the Softmax Sane

A PRACTICAL ASIDE YOU WILL BE GRATEFUL FOR the first time you implement this yourself, because it bites everyone once. The softmax exponentiates its logits — and if a logit happens to be large, say 1000, then e^{1000} overflows to infinity, and your distribution becomes a vector of NaNs, a silent catastrophe. The cure exploits a small invariance: the softmax is unchanged if you *subtract the same constant from every logit* before exponentiating, because that constant factors out of numerator and denominator alike and cancels. So the universal recipe is to subtract

the maximum logit from all of them first:

$$\text{softmax}(z)_i = \frac{e^{z_i - \max_j z_j}}{\sum_k e^{z_k - \max_j z_j}}.$$

Now the largest exponent is exactly zero, $e^0 = 1$, nothing overflows, and the answer is bit- for-bit identical to the naïve formula. Every production softmax on earth does this, and the closely related *log-sum-exp* — $\log \sum_i e^{z_i}$, computed with the same max-subtraction trick — is the numerically stable way to add the surprises of Act I without underflow or overflow.

Surprise, and Why It Must Start at Zero

WE CAN NOW MAKE BELIEFS. But a belief is worthless until the world tests it, and the test has a feeling attached: *surprise*. Surprise is the raw sensation out of which we will build the entire machinery of learning, so let us get its shape exactly right, and let us do it with weather.

Suppose this morning you believed, with 99% confidence, that it would rain — and it rained. How surprised are you? Barely at all. You knew. Now suppose instead you had judged rain a 1% long shot — and it poured. Now you are astonished. The lesson is that surprise is a *decreasing* function of the probability you assigned to the thing that actually happened. The more confidently you expected it, the less it startles you when it arrives; the more you dismissed it, the harder it hits.

What is the simplest function that decreases as p grows and blows up as $p \rightarrow 0$? A student's first, honest guess is the inverse, $1/p$. And it is a good guess! As p shrinks toward zero, $1/p$ shoots to infinity — the 1% rain that ruins the picnic gives a surprise of 100, which feels right. But the inverse has one small, fatal flaw, and finding it is the whole game. Consider absolute certainty: $p = 1$, the event you were *sure* of, and it happens. Your surprise should be exactly *nothing* — zero. But $1/p = 1/1 = 1$. The inverse insists you remain one unit surprised by the utterly inevitable. Its range is $[1, \infty)$ when we need $[0, \infty)$. The floor is in the wrong place.

So we ask a sharper question: how do we keep the lovely “blows up near zero” behaviour of $1/p$, while pulling the floor down to zero at $p = 1$? The easiest repair in all of mathematics: take the logarithm. Define surprise as

$$S(p) = \log \frac{1}{p} = -\log p.$$

There is a small poetry here worth pausing on: $\text{lse}(z) = \log \sum_i e^{z_i}$ is a *smooth maximum* — it tracks the largest logit but rounds the sharp corner of the true max into something differentiable. The softmax is precisely its gradient. So the smooth-max and the soft-argmax are derivative and function of one another, and both will reappear, by name, inside tonight's Hopfield energy. The cat leaves footprints everywhere.

Notice we only ever score the probability you placed on the outcome that *actually occurred*. The rest of your distribution — your beliefs about the events that did *not* happen — costs you nothing today. This is the seed of the negative log-likelihood: it grades you only on the mass you put on the truth.

We met this number on Saturday, in the BERT card game — the chips you placed on the missing word, scored by $-\log_2 p$. That was not a coincidence and not a warm-up. That game *was* this equation, played with your hands. Today we learn why it is the right equation, and not merely a convenient one.

Average NLL has a second name you should hold next to the first: *cross-entropy*. They are the same quantity. When someone says “we trained with cross-entropy loss,” they mean “we made the model minimise its average surprise on the training data.” Two names, one number — and you built it from a rainy afternoon.

Check the two ends. At $p = 1$: $\log 1 = 0$. Perfect — certainty, fulfilled, costs no surprise at all. At $p \rightarrow 0$: $-\log p \rightarrow +\infty$. Perfect — the impossible, made real, is infinitely surprising. The logarithm has taken the right-shaped curve and slid its floor to exactly where intuition demanded. This single quantity, $-\log p$, is *the* atom of information theory; in base two it is measured in *bits*, and it is the literal answer to “how many yes/no questions, on average, would I have needed to pin this outcome down?”

From One Surprise to Many: the Negative Log-Likelihood

ONE EVENT GIVES ONE SURPRISE. But a model does not face one event; it faces a whole corpus — thousands of weather days, millions of missing words. How surprised is it *in total*? Here the logarithm pays a second dividend that we got for free. Because surprise is now a logarithm, the total surprise over many independent events is simply the *sum* of the individual surprises:

$$S_{\text{total}} = \sum_k -\log p_k = -\sum_k \log p_k.$$

This is the **negative log-likelihood**, the NLL — the accumulated surprise of a model across all the evidence it was asked to predict. Averaged over the corpus rather than summed, it is what you will see logged on every training dashboard you ever watch, falling — you hope — epoch by epoch. A model *learning* is, in the most literal possible sense, a model becoming *less surprised* by the world.

Make it concrete with five predictions. Suppose, on five successive missing words, your model placed probabilities 0.5, 0.9, 0.1, 0.8, and 0.6 on the words that actually turned out to be correct. The per-event surprises in bits are $-\log_2 0.5 = 1.00$, $-\log_2 0.9 = 0.15$, $-\log_2 0.1 = 3.32$, $-\log_2 0.8 = 0.32$, and $-\log_2 0.6 = 0.74$. Sum them: 5.53 bits of total surprise; average them: about 1.11 bits per word. Notice that the third event — the word your model judged a 10% long shot, which happened anyway — contributes 3.32 bits, more than the other four put together. This is the signature of the negative log-likelihood, and it is a feature, not a bug: the loss is dominated by the cases where the model was *confidently wrong*, so the gradient shouts loudest exactly where the model most needs to change its mind. An average surprise that falls from 1.11 bits toward 0.3 over training is the whole story of learning, told in one number.

Pause here and admire what we have, because it is a complete little engine. The softmax takes raw logits and makes a belief, a p for

each candidate. The negative log-likelihood takes that belief, looks at what actually happened, and returns a single number: how surprised the belief left us. And because the whole pipeline — exponentiate, normalise, take a log, sum — is smooth and differentiable, gradient descent can take the total surprise and push it downhill, nudging the logits, and through them the millions of weights that produced the logits, in the direction that would have made the truth less surprising next time. Softmax proposes; NLL disposes; gradient descent learns. That is the whole of supervised learning in one sentence, and you have now built every piece of it.

Entropy, Cross-Entropy, and Perplexity

BEFORE WE LEAVE SURPRISE BEHIND, three words deserve to be pinned down, because you will meet them on every dashboard and in every paper, and they are all just this morning’s surprise, averaged and re-dressed. Suppose the world really does draw its outcomes from some true distribution p , and consider how surprised a *perfectly calibrated* observer — one whose beliefs exactly match p — would be, on average. That average surprise is the **entropy**:

$$H(p) = -\sum_i p_i \log p_i = \mathbb{E}_p[-\log p].$$

Read it as exactly what it is: the expected value of $-\log p$, the average surprise you *cannot avoid* even with perfect beliefs, because the world is genuinely uncertain. A fair coin has an entropy of one bit — one unavoidable yes/no question per flip. A two-headed coin has entropy zero — nothing to ask, nothing to learn. Entropy is the irreducible floor of surprise.

But you are not perfectly calibrated — you hold some model q , not the truth p . How surprised are *you*, on average, when the world draws from p but you scored it with q ? That is the **cross-entropy**,

$$H(p, q) = -\sum_i p_i \log q_i,$$

and it is, term for term, the average negative log-likelihood we just built — now you see why the two names are interchangeable. Minimising cross-entropy means making your beliefs q as unsurprised as possible by the truth p ; it bottoms out exactly when $q = p$, when your model has become the world. Training is the slow descent of this single number.

AND THE THIRD WORD TIES IT ALL BACK to Saturday’s table. Take your

WHY? This matters for RAG more than it first appears. Every embedding model you will put in front of Qdrant was trained by exactly this loop — a softmax over candidate matches, scored by negative log-likelihood, walked downhill. When a retriever fails at midnight and returns the wrong clause, it is not being capricious: it is returning the answer that minimised *its* surprise on *its* training distribution, which may not be yours. To debug retrieval you must think like the loss that built it.

Entropy is the surprise you would feel with *correct* beliefs; cross-entropy is the surprise you actually feel with *your* beliefs. The gap between them — always non-negative, zero only when you are exactly right — is the *Kullback–Leibler divergence*, and it is the truest sense in which training “moves your model toward the truth.” We will meet it again in Act III, measuring the distance between distributions on a curved surface.

average surprise in bits and raise two to its power: that is the **perplexity**,

$$\text{PPL} = 2^H.$$

It has a wonderfully concrete meaning: the *effective number of equally-likely options* you are choosing among. A model with an average surprise of three bits is as confused as someone guessing uniformly among $2^3 = 8$ possibilities. In the card game, a clueless player spreading chips evenly over ten words scores $\log_2 10 \approx 3.32$ bits, a perplexity of ten — the whole vocabulary, no narrowing at all. A fluent English player scored near zero bits, a perplexity near one — effectively certain. Your three decks, Tibetan to Spanish to English, walked your perplexity from ten toward one, and that curve *is* the curve a language model walks down across its training. When you read that “the model reached a perplexity of 12”, you now know precisely what was claimed: faced with the next word, it was about as uncertain as a fair twelve-sided die.

The Gradient the Softmax Allows

ONE TECHNICAL GRACE NOTE that explains why this particular pair — softmax in front, negative log-likelihood behind — became the universal coupling, used in virtually every classifier on earth. It is not only that each piece is sensible. It is that when you compose them and ask for the gradient — the thing gradient descent actually needs — an almost miraculous simplification occurs. Let the true answer be a one-hot vector y (a one on the correct class, zeros elsewhere), let $p = \text{softmax}(z)$ be the predicted distribution, and let the loss be the cross-entropy $-\sum_i y_i \log p_i$. The gradient of that loss with respect to the logits is simply

$$\frac{\partial \mathcal{L}}{\partial z} = p - y.$$

Stop and admire how clean that is. The signal that flows back to teach the network is just *predicted minus actual* — the error itself, naked, with no awkward factors and no vanishing. If the model put 0.7 on the right class, the gradient there is $0.7 - 1 = -0.3$, a gentle pull upward; if it put 0.9 on a wrong class, the gradient there is $0.9 - 0 = 0.9$, a firm shove down. The exponential of the softmax and the logarithm of the loss were, in a sense, *built for each other* — the log undoes the exp at exactly the right moment — and their child is a gradient of disarming simplicity. This is the engineering reason the cat and its judge are always seen holding hands: together they are not just correct but *kind to the optimiser*.

This cancellation is also why the softmax must be *soft*. The hard argmax has a gradient of zero almost everywhere — step off the boundary and nothing changes — so it teaches the network nothing. The softmax leaks a little probability everywhere, so there is always a gradient to follow. Differentiability is not a nicety here; it is the whole reason learning is possible at all.

Is a 0.7 Really a 70%? A Word on Calibration

A CAUTION, BEFORE WE TRUST THESE NUMBERS TOO FAR. The softmax *always* hands back something that looks like a probability — it sums to one, each entry sits in $[0,1]$ — but looking like a probability and *being* one are different things. A model is *calibrated* if, among all the times it says “0.7,” the answer really is correct about 70% of the time. Modern deep networks are notoriously *over*-confident: trained hard to minimise negative log-likelihood, they learn to push their winning logit very high, so a reported 0.99 may correspond to a true accuracy of only 0.90. The distribution is sharp where it has no right to be.

Why should you, building retrieval systems, care? Because the moment you start *thresholding* on a score — “only answer if the model is at least 0.8 confident,” “only retrieve if similarity exceeds this cut” — you are betting that the number means what it says, and an uncalibrated model will quietly break that bet. A grounding judge that trusts an overconfident 0.95 will wave through hallucinations; a retriever with a miscalibrated cut-off will drop good documents or admit bad ones. Knowing that the softmax’s tidy numbers are *decisions first and probabilities only if calibrated* is exactly the kind of hard-won wariness that separates a system you can trust at midnight from one you merely hope about.

This is the temperature knob returning as a practical tool. *Temperature scaling* — fitting a single $T > 1$ on held-out data and dividing the logits by it before the softmax — is the standard cure for over-confidence. It cannot change which class wins (recall: a positive divisor never reorders), but it cools the distribution until its confidence matches its accuracy. One number, learned after training, to make the model honest about how sure it is.

Emily’s Objection: Why the Logarithm Is Forced, Not Chosen

NOW, A CONFESSION, AND A DEEPER TRUTH. A careful student — let us give her a name, Emily, and a sharp pencil — has been frowning at the back of the room, and her objection is exactly right. “You performed a sleight of hand,” she says. “You wanted to drag the floor from 1 down to 0, and you reached for the logarithm. But the logarithm is hardly the *only* way to do that. You could have just *subtracted one* from the inverse: $1/p - 1$ is also zero at $p = 1$ and also blows up near zero. Why the log and not the simpler subtraction? You have given me a comfortable intuition, not a reason.”

She is right, and the honest answer is wonderful. To find it, stop thinking about a single event and think instead about *the evidence as a whole*. Suppose you hold beliefs about the probabilities of various events, and the world hands you a sequence of outcomes. If the events are independent, the probability that *your beliefs would have produced exactly this evidence* is the *product* of the individual probabilities. For

Emily is Maximum Likelihood Estimation in disguise, and she is the most important person in the room. Whenever an intuition feels too convenient, send for Emily. She will not let you keep a result you have not earned.

two events it is $p_1 p_2$; for three, $p_1 p_2 p_3$; for a corpus of N , it is

$$L = \prod_{k=1}^N p_k.$$

This product is called the **likelihood** — the plausibility of your model in the light of all the data at once. The best model, says Emily, is the one that makes the observed evidence *most probable*: the model that maximises L . This is Maximum Likelihood Estimation, and it is the bedrock principle under almost all of the learning we do.

Now watch two problems with maximising L directly, and watch a single move solve both. The first problem is computational, and you can feel it in thirty seconds: take a hundred random numbers between zero and one and multiply them together. Try it. The result is so unfathomably tiny — something like 10^{-40} — that your computer's floating-point representation simply gives up and stores it as exactly zero. This is *numerical underflow*, and it is not a rare edge case: a real corpus has not a hundred but billions of factors, and the product underflows to a dead, gradientless zero within the first few hundred terms. You cannot optimise a quantity your machine cannot even represent.

Underflow is not a nuisance to be engineered around; it is the universe telling you that products of many probabilities are the wrong object to compute with. The right object is waiting one step away.

So we ask, as Emily would: is there a transformation that turns this impossible product into something tame, *without* changing where the maximum lies? Consider the logarithm. It is *monotonic*: as x increases, $\log x$ increases, always. Therefore the value of the parameters that maximises L is the *same* value that maximises $\log L$ — moving to log-space slides the whole landscape up and down but never moves the location of its peak. And the logarithm has the one algebraic property we are desperate for: it turns a product into a sum.

$$\log L = \log \prod_k p_k = \sum_k \log p_k.$$

The unrepresentable product of a billion tiny numbers becomes a perfectly civilised sum of a billion modest, negative logarithms — a number near -30 instead of a number near zero, a quantity every float on earth holds with ease, and a sum whose gradient is a clean sum of gradients. Maximise $\log L$; or, since optimisers conventionally descend rather than climb, minimise its negative, $-\log L = -\sum_k \log p_k$.

AND THERE SHE IS AGAIN. $-\sum_k \log p_k$ is precisely the negative log-likelihood we built from rainy afternoons a page ago. Emily and intuition have arrived, by completely different roads, at the same door. This is the answer to her objection, and it is decisive. Why the logarithm

and not the subtraction $1/p - 1$? Because $1/p - 1$ does *not* turn the product-likelihood into a sum — it leaves you stranded with an unrepresentable product and no way down the hill — whereas the logarithm does, and, up to an overall choice of base, it is the *only* continuous function that converts products into sums. The intuitive demand — that independent surprises should *add* — and the rigorous demand — that the likelihood product become a computable sum — are revealed to be *the same demand*. The log was never a convenience. It was forced all along, and Emily made us prove it.

Act II: The Lookup That Learns

THE CAT RETURNS. In Act I the softmax civilised a vector of logits into a decision. In this Act we will watch the very same softmax do something that looks, at first, completely different: it will let a word *look up its own meaning* in a dictionary. And yet by the end you will see it is the identical machine — proximity scores in, a normalised distribution out, a blended answer retrieved. Attention is softmax wearing the costume of a librarian. We approach it not through the famous, forbidding equation but through something every one of you has done a thousand times: searching for a word you cannot quite spell.

Four Exercises in Fuzzy Lookup

LAST WEEK WE BUILT AN INTUITION FOR ATTENTION as “a context giving differential emphasis to its parts”. Today we make that precise, and we begin with a dictionary. Imagine the Merriam-Webster website. You type a word; if it exists, you get its meaning. But what if you type a word that is *not* in the dictionary? A good search does not simply fail — it finds the nearby words it *does* know, and offers their meanings, weighted by how close each is to what you typed. This act of *fuzzy lookup* is, I will argue, the entire idea of attention. Let us do four of them, by hand, and pay attention to our own minds as we do.

Exercise 1 — high-confidence match. You type *recieve*. It is not a word, but the three nearest things the dictionary knows are *receive*, *relieve*, and *recipe*. You have one unit of attention to spread across them. How do you divide it? Almost everyone lands near $[0.95, 0.04, 0.01]$ — a near-certain bet on *receive*, a whisper toward *relieve*, a trace toward *recipe*. And would it be sensible, then, to take the *meaning* of your typo to be mostly the meaning of *receive*, with the faintest tint of the others?

This is the defining property of the logarithm: $\log(ab) = \log a + \log b$. A theorem (Cauchy’s functional equation) makes it precise — the only continuous functions satisfying $f(ab) = f(a) + f(b)$ are the logarithms. So the log is not *a* choice among many; it is *the* bridge from the multiplicative world of independent probabilities to the additive world of gradient

descent

WHY? Hold on to the shape of this argument, because it is the template for the whole course. A soft intuition (“surprise should start at zero”) and a hard principle (“maximise the likelihood, and survive underflow”) turn out to be two views of one mountain. When the two roads meet, you have found something true rather than something merely useful. This is the difference between knowing a formula and *owning* it.

Keep a running tally of the language we are quietly building. The thing you *type* is a **query**. The dictionary words you *match against* are the **keys**. The *meanings* you retrieve are the **values**. Query, keys, values — *Q, K, V*. By the fourth exercise the famous triple will feel like an old friend.

Of course. You have just computed a softmax over proximity scores and retrieved a blended value. You did not know you were doing attention. You were.

Exercise 2 — split attention. Now you type `accross`. Forgive the deliberate typo; the candidates are `access`, `across`, and `actress`. This time the mind does not snap. The scores come out more like $[0.40, 0.35, 0.25]$ — a genuine three-way hedge, because the query is honestly ambiguous between them. Feel the difference from Exercise 1: the *same* machinery, but the *shape* of the output distribution now reflects real uncertainty rather than near-certainty. The softmax did not become less confident on a whim — *the evidence* was more evenly balanced, and the distribution faithfully reported it. A good attention is honest in exactly this way.

Exercise 3 — the keystone. You type `teh`. The candidates are `the` (a transposition of two letters), `ten` (one letter changed), and `tea` (one letter changed). Now stop, because something subtle and beautiful is about to happen. Measured by raw edit distance — the number of keystrokes to repair the typo — all three candidates are roughly *equidistant* from `teh`. A naive system that scored only by edit distance would shrug and return $[0.33, 0.33, 0.33]$. But you did not. You almost certainly gave something like $[0.85, 0.10, 0.05]$, overwhelmingly favouring `the`. Why? Because `teh` is one of the most common typos in the English language: it is what your fingers produce when they fire `t-h-e` a hair out of order, and they do that constantly. `teh`→`ten` or `teh`→`tea` as a slip is far rarer. Your score encodes not the *geometry* of the keyboard alone but the *statistics of human error* — a prior, learned from a lifetime of reading and mistyping. This is the precise sense in which a trained attention is richer than any hand-coded distance: the dot product $\langle q, k \rangle$ that scores a query against a key is itself *learned*, so it can discover that `the` deserves the mass even when a ruler says all three are equally far. Edit distance is geometry; attention is geometry *plus* everything the model has ever learned about which confusions are likely. Sit with that. It is why the mechanism had to be trainable, and it is the reason the next decade happened.

Exercise 4 — structure and stem. Finally you type `runnign`. The candidates are `running`, `run`, and `ruin`. Most of you give roughly $[0.85, 0.10, 0.05]$, and — more interesting than the numbers — I want you to notice *why you chose these three candidates in the first place*. You recognised a transposed ending (`-ign` for `-ing`), you saw the shared stem `run`, and you let `ruin` in only as a grudging long shot on letter overlap. You were doing morphology — stemming, prefix-matching — without being asked. Real subword tokenizers and real attention heads rediscover exactly these regularities. You are not learning an alien mechanism today. You

This is the single most important exercise of the afternoon. If attention were merely “similarity”, edit distance would settle the matter and all three would tie. That it does *not* tie tells you attention scores are *learned priors*, not fixed distances — exactly the thing a transformer trains, and exactly the thing a frozen distance metric can never capture.

are learning the name for something your own mind has always done.

THE LESSON, STATED ONCE AND FOR KEEPS. When an exact match fails, you look at the *nearby keys* (K) and their *meanings* (V), and you weight the meanings by each key's *proximity to your query* (Q). In symbols, the retrieved meaning is a sum of values weighted by query-key proximity, and the weights are a softmax so that they form an honest distribution. That is the core insight of *Attention Is All You Need*, stripped of its notation and handed back to you as something you already knew how to do. Everything that remains is making the words into numbers so a machine can play the same game — and that is the next, unavoidable step.

Before the Vector: Breaking Text into Tokens

WE HAVE BEEN SPEAKING OF “A WORD BECOMING A VECTOR” as if a word were the natural atom of text. It is not, and the gap is worth one careful subsection, because it explains a great many things you will otherwise find baffling — why models stumble on rare names, why they count letters poorly, why a typo like `runnign` is not as alien to them as it looks. Before any vector exists, raw text must be chopped into units the model knows, and those units are *tokens*, not words.

A modern tokenizer learns its vocabulary from data by a procedure like *byte-pair encoding*: start from individual characters and repeatedly merge the most frequent adjacent pair into a new unit, until you have a vocabulary of some tens of thousands of *subword* pieces. Common words survive as single tokens (`the`, `river`); rare or complex ones shatter into familiar fragments (`tokenization` might become `token` + `ization`; an unusual surname might dissolve into a handful of syllables). Each token in the vocabulary owns a row in a giant lookup table — the *embedding matrix* — and that row is its initial vector, the raw, context-free location it occupies before attention has said a word.

So the true pipeline, stated once and exactly: text is split into subword tokens; each token is looked up to its row in the embedding matrix, giving a context-free vector; positional information is added so order is not lost; and *then* the stack of attention layers we are about to build turns those raw vectors into context-aware ones. When we say “the word `bank` becomes a vector,” we are compressing these first steps for the sake of the story. Keep the full chain in mind, though — it is the reason the geometry has anything to work *on*.

This is the quiet reason subword structure matters so much. Because `running`, `runner`, and `runs` share the token `run`, they begin life already neighbours, sharing a fragment and therefore a gradient. Our morphology exercise — spotting the stem in `runnign` — was not a metaphor; it is, more or less, what the tokenizer does for a living.

Why Machines Must Speak Machineese

This is the quiet reason the whole field looks the way it does. Not aesthetics, not convention — the optimiser’s need for small steps. Continuity is not a luxury we chose; it is a precondition gradient descent imposed on us.

The simplest manifold is flat Euclidean space, $\mathbb{R}^n$, and it is where almost all embeddings live. But it is not the only choice. Later in your journey you will meet *hyperbolic* embeddings — the Poincaré ball — whose negative curvature gives exponentially more room near the boundary, perfect for representing trees and hierarchies. The manifold can be chosen to fit the shape of the meaning. File that away.

LAST WEEK WE ASSERTED that machines reason in vectors — “machineese” — and that text, to a machine, is a set of locations on the surface of a high-dimensional globe. We took it on faith. Today we *earn* it, because there is a genuine reason, and the reason is gradient descent itself.

Suppose, early in training, a model holds a word x and believes it sits closest in meaning to a word y . Suppose we know better — the truth is that x belongs near z . Learning must move x *away* from y and *toward* z . But here is the difficulty: gradient descent only ever makes *infinitesimal* moves. It does not leap; it nudges, by a tiny step proportional to the gradient, ten thousand times over. So we need a space in which x can be nudged *a little* — a space where there exist positions *between* the meaning of one word and the meaning of another.

Now ask: can we make such a nudge in the *discrete* world of the lexicon? Between `bat` and `ball`, is there a word in the dictionary meaning “99% bat, 1% ball”? There is not. The lexicon is a scatter of isolated islands with nothing but void between them; you cannot stand a hair’s breadth off the island of `bat`. A discrete vocabulary offers gradient descent no place to put its small steps, and so it offers *no gradient at all*. But the surface of a globe — a smooth, continuous *manifold* — offers infinitely many positions between any two points. On a manifold you can slide x a whisker away from y and toward z , and tomorrow a whisker more. *That* is why meaning must become geometry: because learning is gradient descent, gradient descent needs small steps, and only a continuous manifold has room for them. The discrete word was always a prison; the vector is the parole.

So the machine, denied the dictionary of discrete words, builds itself a *geometric* dictionary instead: every word becomes a vector — a location — and so does every query. The fuzzy lookup we did by hand a moment ago is now a computation the machine can actually perform and, crucially, *differentiate*: proximity becomes a dot product, the weighting becomes a softmax, and the whole librarian’s act becomes a smooth function the optimiser can push downhill. Machineese is not an affection. It is the only language in which learning by gradients can be spoken.

Self-Attention: a Word Looks Up Its Own Meaning

NOW THE MOVE THAT CHANGED EVERYTHING. In the dictionary exercises, the keys were the whole of Webster’s — an external reference book.

But consider the sentence

She swam across the river to the other bank.

and fix your eye on the word *bank*. In isolation, *bank* is hopelessly ambiguous: it is a place that keeps money and it is the edge of a river, and a static dictionary must list both and commit to neither. How does *bank* discover *which* bank it is, here? Not by consulting the comprehensive Webster's — it does not need the whole language. It needs only the *other words in its own sentence*. It can look partly into the mirror — its own intrinsic, context-free meaning — and partly at its neighbours: *swam*, *river*, *across*. Those neighbours scream “water!”, and *bank* resolves, settling onto the riverbank and away from the vault.

So *bank*'s “dictionary” shrinks from all of English to just the words of its own sentence: [*she*, *swam*, *across*, *the*, *river*, *to*, *the*, *other*, *bank*]. The word forms a query from itself, scores that query against the key of every word in the sentence (including itself — the mirror), turns those scores into a softmax distribution, and reads back a blended value: a new, *contextualised* meaning for *bank*. Written as the sum we have been building toward all afternoon,

$$\text{ctx_meaning}(\text{bank}) = \sum_i s_i v_i,$$

where v_i is the value — the “meaning” — carried by word i of the sentence, and s_i is the attention this word pays to word i . Because the river words win most of the mass, the contextual meaning of *bank* is pulled firmly toward the water. The quest is over: the word has found out who it is by asking its neighbours.

And the scores themselves? The most natural measure of “how aligned are this query and this key” is the dot product, $\langle k, q \rangle$ — large when the two vectors point the same way, small when they are orthogonal, negative when they oppose. Pass those dot products through a softmax and you have the weights s_i . So, almost completely,

$$s_i = \text{softmax}\left(\frac{\langle k_i, q \rangle}{\sqrt{d}}\right).$$

There it is — the cat, grinning at us from inside the most famous equation in modern AI. But wait: what is that $\sqrt{d}$ doing in the denominator, dividing the scores before the softmax sees them? Hold that breath. We have a knob, met this very morning, that divides logits before a softmax and controls whether the distribution *snaps* or *shrugs*. We called it temperature. I will say no more until the evening — but if some of you have already guessed what $\sqrt{d}$ secretly is, you have found tonight's deepest rabbit hole on your own.

This is the resolution of the polysemy problem we raised last week with *monarch* (butterfly or king?) and *apple* (fruit or company?). A static embedding gives one frozen vector per word; self-attention gives a *contextual* one, re-computed fresh for every sentence. The same word, two sentences, two different vectors. Context is the disambiguator.

Why *bank* and not, say, *swam*? Because *bank* is the word that *needs* the context — it is genuinely two-faced. Always demonstrate a mechanism on the example that would fail without it. A word with only one meaning learns nothing from its neighbours; a polyseme is saved by them.

One honest simplification I owe you. In a real transformer the query, key, and value are not the raw word vector but three different *learned projections* of it — the word is asked “what are you looking for?” (Q), “what do you offer as a match?” (K), and “what meaning do you contribute?” (V), and it answers each with a different linear map. This is why a single mechanism can play all three roles at once.

Several senses at once. One more refinement, lest you leave thinking attention has a single opinion. Real attention is *multi-headed*: the lookup runs several times in parallel, each “head” projecting into its own subspace and attending for its own kind of relationship — one head might track grammatical subject, another long-range coreference, another local idiom. The heads’ retrieved values are concatenated. Think of it as consulting several specialist dictionaries at once and stitching their answers together. We will not belabour it today, but do not carry away the picture of a lone, single-minded lookup.

One thing we are deliberately ignoring. You may have noticed that the sum $\sum_i s_i v_i$ has no idea what *order* the words came in — shuffle the sentence and the attention weights are unchanged. That is a real and deliberate hole: raw attention is permutation-blind, and word order obviously matters (“dog bites man” is not “man bites dog”). The repair is *positional encoding*, a way of stamping each token with its place in the line. Ask me about it at the break — how a clever choice of sines and cosines lets the model feel position — and why we could get away with ignoring it for one clean afternoon.

Query, Key, Value: the Three Questions a Word Asks

LET US SLOW DOWN ON THE TRIPLE Q, K, V , because it is the part students nod along to and then cannot reconstruct a week later. The confusion always comes from imagining that a word *is* its query, or *is* its key. It is neither. A single word vector is asked to play three different roles in the lookup, and it answers each with a *different learned projection* of itself — three small matrices, W_Q, W_K, W_V , that the training has shaped. Think of the three roles as three questions the word is forced to answer about itself.

$q = W_Q x$ — “What am I looking for?” The query is the word in its seeker’s costume, broadcasting the kind of context it needs in order to resolve itself. *bank*, unsure of its sense, sends out a query that effectively asks: *is there water near me, or money?*

$k = W_K x$ — “What do I offer as a match?” The key is the word advertising what it is good for matching *against*. *river* offers a key that says *I am about water*, so that any query hunting for water will resonate with it.

$v = W_V x$ — “What meaning do I contribute, if chosen?” The value is the payload — the actual semantic content a word pours into the blend

once it has won some attention. Keys decide *how much*; values decide *what*.

The separation of key from value is the subtle, powerful part, and it is worth a sentence of its own. A word can be a strong *match* for a query while contributing a meaning quite different from the thing that made it match — just as, in a library, the catalogue card you search by (the key) is not the book you carry home (the value). Splitting “what makes me findable” from “what I actually say” is exactly what lets attention retrieve nuanced, blended meanings rather than echoing back the search terms.

So when you see $QK^\top$ in the equation, picture every word’s “what am I looking for” being dotted against every word’s “what do I offer” — a full grid of resonances — and when you see the final multiply by V , picture each word collecting its proximity-weighted share of everyone’s contributed meaning. The three matrices W_Q , W_K , W_V are what the model *learns*; the rest of the equation is the fixed choreography we built by hand from typos.

A Worked Attention, End to End

NUMBERS MAKE BELIEVERS, so let us run the entire mechanism once, by hand, on a miniature. Take *bank* resolving itself against just two context words, *river* and *money*, in a toy two-dimensional space. Let *bank*’s query, after its projection, be $q = (1.0, 0.0)$ — “I am seeking the water-or-money axis, and I lean water.” Let the keys be $k_{\text{riv}} = (1.0, 0.2)$ and $k_{\text{mon}} = (0.1, 1.0)$, and the values — the meanings each would contribute — be $v_{\text{riv}} = (0.9, 0.1)$ (a “water” meaning) and $v_{\text{mon}} = (0.1, 0.9)$ (a “finance” meaning).

Step one, score. Dot the query against each key:

$$\langle q, k_{\text{riv}} \rangle = (1)(1.0) + (0)(0.2) = 1.0, \quad \langle q, k_{\text{mon}} \rangle = (1)(0.1) + (0)(1.0) = 0.1.$$

Step two, scale. Divide by $\sqrt{d} = \sqrt{2} \approx 1.414$, giving scores 0.707 and 0.071. *Step three, softmax.* Exponentiate — $e^{0.707} \approx 2.028$, $e^{0.071} \approx 1.074$ — sum to 3.102, and normalise: the weights are

$$s_{\text{riv}} = \frac{2.028}{3.102} \approx 0.654, \quad s_{\text{mon}} = \frac{1.074}{3.102} \approx 0.346.$$

Step four, retrieve. Blend the values by these weights:

$$\text{ctx}(\text{bank}) = 0.654 v_{\text{riv}} + 0.346 v_{\text{mon}} = (0.589 + 0.035, 0.065 + 0.311) = (0.62, 0.38).$$

A tiny worked picture, in two dimensions, for the feel of it. Suppose *bank* produces a query $q = (1, 0)$ meaning “I seek water.” Suppose *river* has key $k_{\text{riv}} = (0.9, 0.1)$ and *money* (imagined present) has key $k_{\text{mon}} = (0.1, 0.9)$. The dot products are $\langle q, k_{\text{riv}} \rangle = 0.9$ and $\langle q, k_{\text{mon}} \rangle = 0.1$. Softmax those two scores and the mass pours onto *river*; the retrieved value is mostly *river*’s, and *bank* settles on water. Change the query to “I seek money,” $q = (0, 1)$, and the very same machinery sends the mass the other way. One mechanism, the sense decided entirely by what the word was looking for.

Notice the temperature at work even here. Had we *not* divided by $\sqrt{2}$, the raw scores 1.0 and 0.1 would have produced sharper weights, roughly 0.71/0.29 — a colder, more committed split. In two dimensions the effect is mild; at $d = 768$ it is the difference between a healthy soft blend and a brittle snap onto a single word. Hold that thought until the evening.

This is the quiet hinge between this morning's mathematics and this afternoon's engineering. Attention makes *token* vectors; retrieval needs a *sentence* vector; pooling is the bridge, and forgetting it leaves a chasm a sharp student will fall into.

The contextual meaning of `bank` has landed nearer the water meaning (0.9,0.1) than the finance meaning, exactly as the river context demanded — but it carries a real tint of finance, because the softmax never zeroes anything out. That single retrieved vector, (0.62,0.38), is what flows onward as `bank`'s meaning *in this sentence*. Change the query to lean the other way and the same four steps would deliver a finance-tinted vector instead. This is the whole machine, and you just turned its crank.

The Missing Rung: from Words to a Single Vector

THERE IS A RUNG IN THIS LADDER that is easy to skip and fatal to omit, so let us set it firmly in place. Self-attention gives us a contextual vector for *every word* in a chunk — nine words in, nine context-aware vectors out. But when we go to build a retrieval system, what do we put into Qdrant? Not nine vectors per chunk — *one*. A search index wants a single point per document so that a single query point can find it. How do we collapse nine contextual word-meanings into one chunk-meaning?

The operation is called *pooling*, and the workhorse is the gentlest thing imaginable: *mean-pooling*, the simple average of the token vectors. (An older recipe took only the vector of a special [CLS] token; mean-pooling reliably beats it for retrieval, because averaging listens to every word rather than betting everything on one.) The average of the nine context-aware vectors is the chunk's embedding — the single location on the globe that we will index, and against which incoming queries will be compared. So the full pipeline now stands complete in your mind: tokens become vectors, attention makes those vectors context-aware, pooling fuses them into one chunk vector, and *that* is what lives in the database. Hold this rung. In Week 3, when we agonise over *where to cut* a document into chunks, the whole anxiety will be about what this average ends up meaning.

The Famous Equation, and a Glance Back at Seq2Seq

WE ARE FINALLY READY to look the famous equation in the eye — the one that, when it appeared in 2017, read like a spell intelligible only to a small cabal of initiates. Here it is, in full:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V.$$

And now, after an afternoon of building it ourselves, it should read not as a spell but as a sentence you could speak. $QK^\top$ is every query scored against every key by dot product — a whole grid of proximities computed at once, in one matrix multiply. Dividing by $\sqrt{d_k}$ is the temperature, keeping the scores from snapping too hard (more tonight). The softmax turns each row of proximities into an honest distribution. And multiplying by V retrieves, for each query, the proximity-weighted blend of values — the contextual meaning. Query, key, value; score, normalise, retrieve. The equation is the four fuzzy-dictionary exercises, written in linear algebra and done in parallel for every word at once.

AND ONE LAST RECOGNITION BEFORE WE REST THE ACT. Recall the encoder–decoder *seq2seq* scaffold we sketched last week: an encoder reads the input into a latent meaning, a decoder writes the output from it, and — in the older designs — a single bottleneck vector had to carry the entire input across the gap. The decoder writes one word at a time, each new word conditioned on everything written so far; written out, the probability of a whole output sequence factorises autoregressively,

$$p(y_1, y_2, \dots, y_T) = \prod_{t=1}^T p(y_t \mid y_{<t}, x),$$

and — look — there is the product of probabilities again, the very object Emily taught us to turn into a sum of log-likelihoods this morning. Training a sequence model *is* minimising the negative log-likelihood of this factorisation, one predicted token at a time. The bottleneck design forced every $p(y_t \mid y_{<t}, x)$ to read the entire input x through one cramped vector, and that was its undoing. Bahdanau’s insight was to let the decoder *attend* back over all the encoder’s states rather than squeeze through one bottleneck — the first attention. Look now at the transformer with today’s eyes: is it not simply this idea taken to its conclusion? The bottleneck dissolved entirely; attention promoted from a patch on seq2seq to the *central organ* of the architecture, applied not only between encoder and decoder but of each sequence *to itself*. “Attention is all you need” was the literal claim — keep the attention, throw away the recurrent scaffolding it was bolted onto, and you lose nothing. The transformer is seq2seq with attention promoted from servant to sovereign.

It is worth remembering how mysterious this looked in 2017, and how ordinary it looks now that you have built it from typos and weather. That is what understanding feels like from the inside: the spell becomes a sentence.

Stacking the Lookups: Layers and the Residual Stream

This is why depth matters. Shallow attention catches local disambiguation; the higher layers compose those resolved meanings into phrases, clauses, and arguments. Empirically, lower layers track syntax and the upper layers track semantics — the model rediscovers, on its own, the linguist’s ladder from form to meaning.

ONE ATTENTION IS A SINGLE ACT OF DISAMBIGUATION, but a transformer stacks dozens of them, and the stacking is where the depth of understanding comes from. Picture *bank*’s contextual vector — the (0.62, 0.38) we just computed — not as a final answer but as a *better question* for the next layer to ask. At layer one, *bank* learned from *river* that it is watery. At layer two, now that every word is a little wiser, it can ask a subtler question and discover, say, that *across* and *other* place it on the *far* side of the river. Meaning is refined in passes, each layer’s contextual output becoming the next layer’s input, a word’s understanding of itself deepening as it climbs.

The vector that carries a token’s evolving meaning up through the layers has a name worth knowing: the *residual stream*. At each layer, attention does not *replace* the token’s vector — it computes an update and *adds* it back: $x \leftarrow x + (\text{attention output})$, and similarly for the feed-forward sublayer. So the stream is a running sum, a bus along which every layer reads the meaning so far, writes a contextual correction, and passes it on. This additive design is what lets gradients flow cleanly down through a very deep stack (the update is a small correction, never a wholesale overwrite), and it is the structure that the new science of *mechanistic interpretability* reads when it claims to “query into a model’s weights” — the phrase I left dangling last week. When researchers find a direction in the residual stream that encodes “this text is in French” or “this entity is a person,” they are reading meaning off exactly this bus. The features we will one day retrieve over, and the features a model thinks in, live in the same geometry. We only glimpse it today; later weeks open the panel.

Guess the Missing Word: BERT in the Body

WE HAVE A MACHINE THAT MAKES BELIEFS (softmax), a judge that scores them (NLL), and a mechanism that lets a word read its context (self-attention). Assemble them and you have the training objective that built the first great encoder, BERT — and rather than define it, we are going to *play* it, because some things must be felt in the hands before they are believed in the head.

The game is called *Guess the Missing Word*. You are dealt cards, each a sentence with one word blanked out, and a small vocabulary of candidate words. Your task: spread ten chips across the candidates according to your belief about the missing word — chips are probability, ten chips is certainty, and a softmax in your skull is doing the spreading. Then the true word is revealed, and you score yourself by $-\log_2 p$, the

surprise of the chips you placed on the truth. Lowest total surprise wins. You played this on Saturday; today you know that the score sheet was a negative-log-likelihood calculation and the chip-spreading was a softmax, so the game is no longer a game — it is a training loop with you as the model.

Two moments in the game repay the whole afternoon. The first is the *zero-chip trap*: place no chips on a candidate, and if it turns out to be the answer, your surprise is $-\log 0 = \infty$ — you lose instantly. This is not a cruel rule; it is the reason real models never assign exactly zero probability to anything (the softmax, with its strictly positive exponentials, cannot), and the reason log-loss punishes confident wrongness without mercy. The second is the *bidirectional* round, where you first see only the left half of the sentence, bet, and then see the right half and bet again. The collapse in your surprise when the right context arrives is, in bits, exactly what *reading both directions at once* is worth — which is the entire architectural argument of BERT over a left-to-right model, made physical on a table in front of you.

AND THEN THE QUESTION that justifies the whole exercise. If a *machine* could play this game well — if, across billions of sentences, it consistently placed most of its chips on the right missing word — what would you be forced to conclude about its grasp of language? You could not dismiss it as memorisation; the sentences are novel. You could not call it luck; the consistency forbids it. You would have to admit it had *learned something real* about how words constrain one another — grammar, selectional preference, world knowledge — because there is no other way to keep the score low. This is the audacious bet of masked language modelling, and it paid off: a model trained only to be unsurprised by missing words turned out to have built, as a side effect, contextual representations so rich that they became the foundation of modern retrieval. The game has a technical name — *masked language modelling*, or, in the older language of psychology, the *cloze* task — and the representations it forges are, quite literally, the embeddings you will spend the rest of this course retrieving over.

Act III: The Geometry of Belief

WE HAVE BUILT THE SPACE AND THE MACHINE THAT FILLS IT. In this final Act we turn to the *shape* of the filled space, and to the practical art of moving through it. We will find that the training game of Act I, left to itself, sculpts a space that is subtly broken — and we will meet the

The decks are dealt in a deliberate order — Tibetan, then Spanish, then English — and the order *is* the lesson. Tibetan (which no one in the room reads) is an untrained model: near the clueless baseline of $\log_2 V$ bits. Spanish is a partially-pretrained model, rescued by cognates. English is a fluent, pretrained model, its surprise near zero. Your own scores, falling deck by deck, trace a *perplexity curve* — the same curve a model walks down during pretraining.

WHY? This is the deepest tie between today's morning and your day job. The encoder you load from **sbert.net** to embed your documents was forged in exactly this fire: a softmax over a vocabulary, scored by negative log-likelihood, played billions of times. Its sense of which sentences "mean the same thing" — the sense you will lean on for every semantic search — is a fossil of how unsurprised it learned to be. When you understand the game, you understand the embedding, and when you understand the embedding, you can finally reason about why retrieval succeeds or fails.

repair we named but did not earn last week. We will learn to see search as motion on a curved surface. And we will end by building something real: a search that, given a story, finds the right photograph for every paragraph.

Anisotropy: When Attraction Is Not Enough

LAST WEEK WE GLIMPSED A FLAW in raw embeddings and named its cure in passing. Now we face it squarely. When a model is trained *only* to predict words — only the masked-word game, only negative log-likelihood — it learns a representation that is powerful but geometrically *lopsided*. All the vectors drift into a narrow *cone*, crammed into one corner of the globe, so that even two unrelated sentences look fairly similar because *everything* looks fairly similar. This pathology is called **anisotropy** — the space is not the same in all directions; it has a strong grain — and it is poison for retrieval, because retrieval lives or dies on the *gaps* between things.

The deep cause is that the language-modelling loss is almost entirely a force of *attraction*. It pulls words that share contexts together, but it is never explicitly told to push anything *apart*. A field of forces with attraction and no repulsion has one stable end state: a single huddled clump. Stars need gas pressure to resist gravity; embeddings need repulsion to resist collapse.

What is missing is a *repulsive* force. And this is precisely what *contrastive learning* supplies. The recipe is almost embarrassingly intuitive: show the model pairs of things that *should* be close (a sentence and its paraphrase; an image and its caption) and pull them together, but *also* show it pairs that should be *far* (a sentence and an unrelated one) and push them apart. Attraction and repulsion, together, do what gravity alone never could: they spread the representations out to fill the sphere, restoring the *isotropy* — sameness-in-all-directions — that makes distances meaningful again. This is the family of methods — Sentence-BERT, SimCSE, and their kin — that turns a raw language model into a genuine *embedding* model fit for search.

The Shape of the Contrastive Objective

LET US WRITE THE REPULSION DOWN, because the equation is more intuitive than its reputation, and — of course — the cat is hiding in it. The oldest form is the *triplet loss*. Take an *anchor* sentence a , a *positive* p that should be close to it (a paraphrase), and a *negative* n that should be far (an unrelated sentence). Ask that the anchor sit nearer its positive than its negative by at least a margin m :

$$\mathcal{L}_{\text{triplet}} = \max(0, \|a - p\|^2 - \|a - n\|^2 + m).$$

Read it as a rule of etiquette enforced by a small fine. If the positive is already closer than the negative by the margin, the loss is zero — no complaint, no gradient. Only when a negative crowds too close, or a positive drifts too far, does the loss switch on and pay out a gradient that *pulls the positive in and shoves the negative away*. This is the missing repulsive force, made literal: attraction to the paraphrase, repulsion from the stranger, balanced by a margin so the space cannot either collapse or fly apart.

The modern, more powerful form is *InfoNCE*, and here the cat is unmistakable. Instead of one negative, gather a whole batch of them, and ask the model to pick the true positive out of the crowd — a classification problem, scored, inevitably, by a softmax over similarities:

$$\mathcal{L}_{\text{InfoNCE}} = -\log \frac{\exp(\text{sim}(a, p)/\tau)}{\sum_j \exp(\text{sim}(a, x_j)/\tau)}.$$

Look at what this is: a softmax over the similarity of the anchor to every candidate, scored by negative log-likelihood of placing the mass on the true positive. It is *the entire morning in one line* — softmax in the denominator, $-\log$ out front, a temperature τ on the scores — now turned to the task of spreading a space rather than classifying a picture. Minimising it pulls each anchor toward its true partner and, through the denominator, pushes it away from every impostor in the batch at once. The same lock and key from Act I, the same cat, now sculpting the very globe you will search across.

THE QUALITY OF THE LESSON depends entirely on the quality of the negatives, and this is the single most practical thing to know when you fine-tune an embedder for your own corpus. *Easy* negatives — a physics sentence paired against an obviously unrelated history sentence — teach almost nothing, because the model already knows they are far apart; the gradient is near zero and the batch is wasted. The instructive negatives are the *hard* ones: documents that look deceptively similar to the right answer but are subtly wrong — the near-duplicate clause from the wrong contract, the regulation from the wrong jurisdiction. Mining these hard negatives, often by using the bi-encoder itself to find its own most embarrassing confusions and feeding them back as negatives, is where most of the real gain in a fine-tuned retriever comes from. A cheap trick that helps for free is *in-batch negatives*: within a training batch, every other example's positive serves as a negative for yours, so a batch of size B gives you $B - 1$ negatives at no extra cost — which is exactly why contrastive training loves large batches. When you later ask “why is my fine-tuned embedder no better than the stock one,” the

The margin m is what stops the trivial solution. Without it, the model could satisfy “positive nearer than negative” by squashing *everything* to a point, where all distances are zero. The margin demands a *real* gap, and a real gap is exactly the isotropy that retrieval needs.

answer is, nine times in ten, that your negatives were too easy.

Seeing the Repulsion: a Demonstration to Run

CLAIMS ABOUT GEOMETRY SHOULD BE SEEN, NOT BELIEVED, so here is a demonstration whose result you will hold in your hands rather than take from me. We take a corpus of sentences from three plainly different subjects — physics, biology, history — and we embed every sentence three times, with three different machines: first a *raw* language model (attraction only, the anisotropic cone); then a generic off-the-shelf sentence embedder downloaded from `sbert.net` (contrastively trained on broad data); and finally a model contrastively fine-tuned on *this* specific three-subject dataset. Three embedders, same sentences, three geometries.

The progression is the moral: raw → generic-contrastive → domain-contrastive is a journey from a use-less huddle, to a usable separation, to a sharp one. It is also, not coincidentally, the journey your own RAG system will take as you move from a stock encoder to a fine-tuned one.

Now we *measure*, because a scatter plot can flatter. For each embedder, compute two collections of cosine similarities: the *within-class* similarities (physics sentence to physics sentence) and the *between-class* similarities (physics sentence to history sentence). Plot each as a histogram. In a *good* embedding space, the within-class histogram sits high (kindred things are close) and the between-class histogram sits low (strangers are far), and — this is the number to watch — the two histograms barely *overlap*. In the raw, anisotropic embedder, the two histograms sit almost on top of one another, both bunched near high similarity: everything looks alike, signal drowned. As you move to the generic and then the domain-tuned embedder, you will *watch the between-class histogram slide left, away from the within-class one*, the overlap shrinking before your eyes. If you want a single number to put on a slide, report the *overlap area* of the two histograms, or a silhouette score: it falls as the space gets healthier. This is what “a good embedding” means, made quantitative — and it is exactly the property your retriever is silently exploiting every time it works.

WHY? This demonstration is the most directly actionable thing in today’s session. The single biggest lever on RAG quality is the embedding model, and “is my embedding model good for *my* data” is not a vibe — it is the overlap between these two histograms on *your* corpus. When you later ask whether to fine-tune an embedder, this is the measurement that answers it. Learn to draw these two histograms and you can diagnose a retrieval system’s foundations in an afternoon.

Two Ways to Use a BERT: Bi-Encoder and Cross-Encoder

THERE ARE EXACTLY TWO WAYS to ask a BERT-like model “how related are these two pieces of text,” and the difference between them organises the entire architecture of a retrieval system, so it is worth drawing sharply. In the *bi-encoder*, you push each text through the model *separately*, pool each into a single vector, and compare the two vectors by cosine. The query never meets the document inside the model; they meet only at the very end, as two points whose angle you measure. In the *cross-encoder*,

you concatenate the query and the document and push them through the model *together*, so that every word of the query can attend to every word of the document at every layer; the model then emits a single relevance score.

The trade is stark and it is all about *when* the comparison happens. The cross-encoder is far more accurate — full attention between query and document catches subtleties a pair of frozen vectors cannot — but it is ruinously expensive at scale: to find the best of a million documents you would have to run the full model a million times, *per query*, because the score cannot be computed until the query is in hand. The bi-encoder is the opposite: every document can be embedded *once*, in advance, and stored in Qdrant; at query time you embed only the query and let the geometry find its neighbours in milliseconds. Cheap, fast, slightly blunt.

The slogan to remember: the bi-encoder *encodes once, compares forever*; the cross-encoder *compares once, must re-encode every time*. This single economic fact is why real systems use both, in sequence.

SO A SERIOUS RAG SYSTEM USES BOTH, in a *cascade*. The bi-encoder casts a wide, cheap net, retrieving perhaps the top hundred candidates from millions by pure geometry. Then the cross-encoder — the expensive, accurate judge — *re-ranks* just those hundred, running its full attention on each query–candidate pair to sort the shortlist precisely. Wide and cheap, then narrow and sharp. You will build exactly this retrieve-then-rerank cascade later in the course, and now you know *why* its two stages are different animals: they are the two ways of using the attention you built this afternoon, traded off against the brutal arithmetic of scale.

Cosine, Dot Product, and the Sphere

A SMALL BUT LOAD-BEARING CLARIFICATION about *how* we measure nearness on this globe, because three quantities are easy to confuse and the confusion causes real bugs. The dot product $\langle u, v \rangle$ measures alignment, but it is contaminated by length: a long vector scores high against everything, simply for being long. The *cosine similarity* removes the contamination by dividing out both lengths,

$$\cos \theta = \frac{\langle u, v \rangle}{\|u\| \|v\|},$$

leaving a pure measure of *direction* — +1 for perfectly aligned, 0 for orthogonal, −1 for opposed — which is exactly what we want when meaning lives in *which way* a vector points, not how long it is. And here is the tidy fact that makes practitioners normalise their embeddings to unit length: once every vector sits on the unit sphere, $\|u\| = \|v\| = 1$, the cosine *is* the dot product, and it is also a monotonic function of

The practical trap: if you forget to normalise, dot-product search silently rewards long vectors, and a handful of high-norm chunks dominate every result list. Many a mysterious retrieval bug is a missing normalisation. On the sphere, length is meaningless by construction, and the bug cannot occur.

This is the doorway to *information geometry*: the study of spaces of probability distributions as curved manifolds, with their own notion of distance (the Fisher–Rao metric) and their own straight lines (geodesics). We will not climb through the doorway today, but I want you to see it is there. A distribution is a point; a family of distributions is a surface; and learning is a trajectory across it. Tonight’s rabbit holes lean on the same view.

the Euclidean distance between the two points. On the unit sphere, the three notions of nearness collapse into one. This is why a vector database can store normalised vectors and rank by a plain dot product, yet you may speak of “cosine similarity” and “nearest neighbour” interchangeably — on the sphere, they agree.

Search as Motion on a Globe

HERE IS THE MESSAGE OF THE DAY, the one sentence to carry out the door. Once meaning is geometry, *search is navigation* — and the surface we navigate is curved. Take an actual globe in your hands, if you have one, because the intuition it gives is worth more than a page of formulae. Our embeddings are normalised to unit length, which means they all live on the *surface* of a hypersphere, not in its interior. The natural notion of “how similar” is the cosine of the angle between two points — which is to say, how far apart they are *along the surface*, the great-circle distance, exactly as the distance between two cities is measured along the Earth’s curve and not by tunnelling through the rock.

Why does this view earn its keep? Because it tells you that the operations you perform in retrieval — and their failure modes — are *geometric* facts, not implementation quirks. Approximate nearest-neighbour search is the art of finding the closest point on the globe without measuring your distance to all of them. Re-ranking is a refinement of the metric near a small neighbourhood. The curse of dimensionality and the concentration of measure from last week are statements about how points distribute on this surface as it gains dimensions. Hold the globe; think “where on the surface, and how far along it,” and a dozen later techniques become obvious responses to the shape of the space rather than a bag of unrelated tricks.

AND THERE IS A DEEPER SENSE in which geometry and this morning are the same study. When the points of our space are not embeddings but *probability distributions* — a model’s beliefs — the natural notion of “how far apart” is no longer the cosine but the *Kullback–Leibler divergence*,

$$D_{\text{KL}}(p \parallel q) = \sum_i p_i \log \frac{p_i}{q_i} = H(p, q) - H(p),$$

and look what it decomposes into: the cross-entropy of q relative to p , minus the entropy of p — the two quantities we built in Act I. The KL divergence is exactly the *excess* surprise you suffer for holding the wrong beliefs q instead of the truth p , and it is zero only when $q = p$. So minimising cross-entropy — the thing every model on earth does

— *is* minimising the KL distance from your model to the world, which *is* walking your belief, step by step, toward the truth across a curved space of distributions. Information geometry is the name for taking that “space of distributions” seriously as a manifold with its own curvature and its own straight lines. The morning’s surprise and the afternoon’s globe were, once again, one subject.

Indexing the Globe: Finding a Neighbour Without Asking Everyone

THERE IS A HARD PRACTICAL PROBLEM lurking under “find the nearest point on the globe,” and it is worth seeing clearly, because it is what a vector database spends its life solving. If you have ten million document vectors and a query arrives, the honest way to find the closest is to compute ten million dot products and take the best — an *exact* search that is correct and far too slow for an interactive system. We need to find the nearest neighbour *without* measuring our distance to everyone, and the willingness to be occasionally, slightly wrong about it is what buys the speed. This is *approximate nearest-neighbour* search, and it is the engine inside Qdrant.

The dominant idea is beautifully geometric: build a *navigable small-world graph* (the HNSW algorithm) over the points, connecting each vector to a handful of near neighbours and a few long-range “shortcut” links. To answer a query you do not scan the globe — you *walk* it: start anywhere, hop to whichever neighbour is closer to the query, and repeat, descending greedily toward the query’s region like water finding a drain. The long-range shortcuts let you cross the globe in a few hops (the “small-world” property, the same six-degrees structure that connects distant strangers in a social network), and the local links let you home in precisely once you are close. A few dozen hops, not ten million comparisons. Once again the geometry is not decoration: the search is *literally* navigation across the surface we have been describing all day, and its speed comes from the shape of the space.

One Space for Words and Pictures: the Picture-Book

WE CLOSE WITH SOMETHING THAT SHOULD FEEL LIKE MAGIC and, after today, will feel instead like an obvious consequence. We are going to build a search over a library of twenty-five thousand photographs — the Unsplash collection, already embedded and indexed for you in Qdrant — and use it to make a *picture-book*: given a written story, find,

Note that KL is *not symmetric*: $D_{\text{KL}}(p\|q) \neq D_{\text{KL}}(q\|p)$. It is a divergence, not a distance — a directed measure of “how surprised p is by q .” The asymmetry is not a defect; it is the reason the order of arguments in a loss function matters, and a source of genuine, consequential design choices later in your machine-learning life.

The trade is explicit and tunable: a knob trading *recall* (did I actually find the true nearest?) against *latency* (how fast?). At one extreme, exact and slow; at the other, lightning-fast and sometimes wrong. Most of the art of operating a vector database at scale is finding the right setting of this knob for your tolerance — and you will tune it yourself later in the course.

Qdrant is a vector database: it stores embeddings and answers the question “which stored points are nearest this query point?” at scale. It is the globe of the last section, made into infrastructure. Everything we have built today is what gets *put into* it; Qdrant is how we move through it quickly.

for each paragraph, the photograph that best fits it. A machine that reads a paragraph of *words* and fetches the right *image* — how?

The secret is a single shared space. A model called *CLIP* was trained contrastively — the very repulsion-and-attraction of two sections ago — on hundreds of millions of *image-caption pairs*, with one rule: pull each image and its true caption together, push mismatched pairs apart. After enough of this, images and the texts that describe them *land in the same place* on a shared globe. A photograph of a child hugging a dog and the words “learning to love an animal” become neighbours — not because the pixels resemble the letters, but because both were dragged to the same coordinates by the contrastive game. And *that* is the whole trick: to find a picture for a paragraph, embed the paragraph as a point, and ask Qdrant for the nearest image points. Text goes in, images come out, because they were never in separate spaces to begin with.

So the picture-book exercise is not a party trick bolted onto the course — it is every idea of the day standing in a row. Contrastive learning built the shared space (Act III); the embeddings are the contextual meanings forged by attention and the masked-word game (Act II); the nearest-neighbour search is motion on the globe; and the softmax-shaped notion of “nearby” that ranks the candidates is the cat, taking its final bow. When your story conjures the right photograph for each paragraph, you will be watching the entire machinery of the day cooperate — and you will have built, with your own hands, a small piece of the thing that made enterprise semantic search possible.

For Those Who Dare: an Evening Down the Rabbit Hole

THE OFFICIAL SESSION IS OVER. What follows is for those who want to see how deep the burrow goes — four descents, each one a place where the Cheshire Cat of the morning turns out to have been something far stranger than a normalising trick. None of this is required. All of it is the reason some of you came.

Rabbit Hole 0: Concentration of Measure, Revisited

LAST WEEK WE MARVELLED that in high dimensions, random points flee to the boundary and distances concentrate — the curse and the blessing as one. Let us put a touch more rigour on it, because it is about to become the key to the deepest connection of the evening. Take two

independent random vectors q and k in d dimensions, each component drawn independently with mean zero and unit variance. Their dot product is a sum of d independent terms, $\langle q, k \rangle = \sum_{i=1}^d q_i k_i$. Compute its statistics directly. The mean is zero, because each term $q_i k_i$ has mean $\mathbb{E}[q_i] \mathbb{E}[k_i] = 0$. The variance, since the terms are independent, is the sum of the variances:

$$\text{Var}\left(\sum_{i=1}^d q_i k_i\right) = \sum_{i=1}^d \text{Var}(q_i k_i) = \sum_{i=1}^d \mathbb{E}[q_i^2] \mathbb{E}[k_i^2] = \sum_{i=1}^d 1 \cdot 1 = d.$$

So the variance of the dot product is exactly d , and its standard deviation — its typical magnitude — is $\sqrt{d}$. This is the quiet engine under both of last week's phenomena: as dimension climbs, the spread of dot products widens like $\sqrt{d}$, and a great many geometric facts that feel paradoxical in high dimensions are really just this one scaling law, $\sqrt{d}$, seen from different angles.

Rabbit Hole 1: Softmax Is Maxwell–Boltzmann

WHY DOES THE SOFTMAX KEEP APPEARING? Here is the first deep answer. There is a hot mug of coffee on the table. Some of its water molecules are sluggish; a few are fast enough to tear free of the surface and escape as vapour. Ask yourself, before any mathematics: if you knew *nothing* about how energy is distributed among the molecules except the total, what is the most honest distribution to guess? The principle that answers this is *maximum entropy*: assume as little as possible, spread your belief as widely as the constraints permit, commit to no order you have not been given.

When you sip the coffee you do not care about the position and momentum of each molecule — an absurd amount of information. You care about one number, the *temperature*, which is sufficient to summarise the whole seething ensemble. So pose the problem formally: among all probability distributions over the energy states E_i of the molecules, consistent with a fixed average energy, which has the maximum entropy? This is a constrained optimisation, and we solve it the way Lagrange taught us. We wish to maximise the entropy $H = -\sum_i p_i \log p_i$ subject to two constraints: that the probabilities sum to one, $\sum_i p_i = 1$, and that the average energy is some fixed value, $\sum_i p_i E_i = U$. Build the Lagrangian, one multiplier per constraint:

$$\mathcal{L} = -\sum_i p_i \log p_i - \alpha \left(\sum_i p_i - 1 \right) - \beta \left(\sum_i p_i E_i - U \right).$$

Remember this number: the dot product of two random d -vectors has a spread of order $\sqrt{d}$. Everything in the next rabbit hole turns on it. We met its reciprocal, $1/\sqrt{d}$, last week in the concentration bounds; we are about to meet $\sqrt{d}$ itself, in a place that will startle you.

Now take the variation — differentiate with respect to a single p_i and set it to zero:

$$\frac{\partial \mathcal{L}}{\partial p_i} = -\log p_i - 1 - \alpha - \beta E_i = 0 \implies p_i = e^{-1-\alpha} e^{-\beta E_i}.$$

The first factor is just a constant; fix it by demanding that the probabilities sum to one, and that constant becomes precisely the normalising denominator. What falls out is the *Maxwell–Boltzmann distribution*, with $\beta = 1/kT$:

$$p_i = \frac{e^{-E_i/kT}}{\sum_j e^{-E_j/kT}}.$$

Look at it. *It is the softmax* — the very equation from this morning’s graph paper — with energy E_i playing the role of (negative) logit and the physical temperature T playing the role of our temperature knob. The cat was never just a normalising trick. The softmax is the maximally non-committal distribution consistent with a set of scores, which is why it surfaces anywhere a system must turn “preferences” into “probabilities” while assuming nothing more than it must.

Rabbit Hole 2: the $\sqrt{d}$ Is a Temperature

This is Jaynes’s great unification: the Boltzmann distribution of statistical mechanics is not a fact about coffee in particular — it is what *maximum entropy under an average-value constraint* always looks like, whether the thing constrained is molecular energy or a neural network’s logits. The softmax is the universe’s default answer to “be as honest as possible given what you know”.

Here is the unification I promised at dawn. The $\sqrt{d}$ that scales attention and the concentration of measure from last week are *the same fact* — both are the $\sqrt{d}$ growth of a sum of d independent terms. Dividing the attention scores by $\sqrt{d_k}$ is undoing, exactly, the dimensional spreading that concentration of measure creates. Last week’s geometry and this afternoon’s equation were the same theorem wearing two hats.

I will confess a small joy here, because it is the joy that made me a teacher. For years I read that $\sqrt{d_k}$ as a footnote — a numerical nicety, a thing one divides by because the paper said so. The day it cracked open into *temperature*, and the temperature into the same $\sqrt{d}$ that governs the geometry of high-dimensional space, an isolated trick became a thread in a single tapestry. That is what I want for you more than any fact: the moment a memorised gesture turns out to have been a law of nature in disguise. Chase those moments. They are the whole reward.

NOW WE COLLECT THE BREATH you have been holding since the afternoon. Why is there a $\sqrt{d_k}$ in the denominator of the attention equation? The paper offered a modest, true reason: it keeps the dot products from growing too large and saturating the softmax. But after Rabbit Holes 0 and 1 we can say something far more exact, and far more beautiful. Recall from Rabbit Hole 0 that the dot product of two d -dimensional vectors has a typical spread of order $\sqrt{d}$. Those dot products are exactly the attention scores. So, *without* dividing by $\sqrt{d_k}$, the scores entering the softmax would have a spread that grows with dimension — and a softmax fed ever-larger spreads, we learned this morning, runs ever *colder*: it snaps hard onto a single key, prematurely, before the model has really weighed its options.

So divide by $\sqrt{d_k}$ and you hold the temperature of the attention softmax *constant* as dimension grows: the scores keep an $O(1)$ spread, the distribution stays appropriately “warm,” and the head can attend softly to several keys instead of snapping onto one.

Read this way, the $\sqrt{d_k}$ is not a numerical stabiliser at all — it is a *thermostat*. It injects exactly enough “heat” to prevent the premature, brittle, single-key collapse we watched happen this morning when we turned the temperature toward zero. The cat, the coffee, and last week’s

high-dimensional globe are revealed to be one animal. Few things in this field are as satisfying as watching a humble $\sqrt{d}$ tie the whole day together.

Rabbit Hole 3: Attention Is Associative Memory

ONE LAST DESCENT, the deepest. We have called attention a fuzzy dictionary lookup. There is a formal name for a system that stores patterns and retrieves the whole from a part: an *associative memory*, and its classical model is the *Hopfield network*. Give an old Hopfield network a noisy or partial pattern and it relaxes, step by step, into the nearest stored memory. Its flaw was capacity: it could reliably store only a handful of patterns before they blurred into one another.

In 2020, Ramsauer and colleagues proved something startling. Store a set of patterns as the columns of a matrix K , and define an energy landscape over the query state q whose wells sit at the stored patterns. The classical Hopfield network used a quadratic energy, and quadratic wells are shallow and few — hence the tiny capacity. The *modern* network uses an energy built from a log-sum-exp of the query’s alignments with the stored patterns,

$$E(q) = -\text{lse}(\beta, K^\top q) + \frac{1}{2}\|q\|^2 + \text{const},$$

and the exponential inside sharpens the wells so dramatically that the network can store *exponentially* many patterns without their blurring together. Now minimise this energy by one step — and the update that moves q toward the nearest stored pattern turns out to be

$$q^{\text{new}} = V \text{softmax}(\beta K^\top q),$$

which, with $\beta = 1/\sqrt{d_k}$, is *identical, term for term, to softmax self-attention*: the same keys, the same query, the same temperature, the same softmax, the same weighted sum of values. In their words, “Hopfield networks is all you need.” Read through this lens, every attention layer is a massive, parallel, content-addressable memory: the keys K and values V are the stored patterns, the query q is a partial cue, the dot products measure resonance between cue and stored content, the softmax is the *fuzzy addressing* that decides how much of each memory to summon, and the weighted sum $\sum_i w_i v_i$ is the retrieved — possibly blended — recollection. A transformer is not metaphorically like a memory. It *is* one, performing billions of fuzzy retrievals to keep updating the meaning of each token as it flows through the layers.

Associative memory is how *you* remember. A bar of a song, a whiff of a kitchen, the first two words of a quotation — a fragment, and the whole floods back. That the transformer’s central operation turns out to be exactly this is one of the more haunting coincidences in the field, if it is a coincidence at all.

WHY? Step back and see what the cat has shown us. The softmax of this morning’s graph paper is the attention of the afternoon is the Boltzmann distribution of the coffee cup is the retrieval rule of a Hopfield memory. One small equation, met first as a way to decide between a cow and a duck, turns out to be the shared skeleton of decision, attention, statistical physics, and memory. This is what we meant, on the very first morning, by saying that a great deal of modern machine learning is one idea in many costumes. Learn to recognise the grin, and the whole menagerie becomes one creature.

A Handful of Confusions Worth Pre-empting

WHEN YOU RETURN TO THIS DOCUMENT in a later week, short on time and trying to remember what today actually established, these are the slippery points most worth having straight. Each is a place a careful person stumbles, and each has a one-line fix.

The softmax is not a maximum. It is a soft *argmax* — a smooth, differentiable pointing-at the largest entry, not the largest entry itself. Its whole reason for existing is that the true max has no useful gradient.

Logits are not probabilities, and softmax outputs are only calibrated probabilities if you check

A tidy 0.9 is a decision; whether it means “90% of the time” is a separate, empirical question, and modern models are usually overconfident until cooled.

Attention is not “similarity”. If it were, equal edit distance would force equal weights — recall *teh*. Attention scores are *learned* alignments through the W_Q, W_K projections, so they encode priors a fixed distance metric never could.

Cosine equals dot product only on the unit sphere. Forget to normalise and the dot product silently rewards long vectors; a few high-norm chunks then dominate every result. Normalise, and the three notions of nearness coincide.

Negative log-likelihood scores only the truth. It grades the probability you placed on the outcome that *actually happened*; the rest of your distribution is free. This is why confident-and-wrong is punished so savagely — the one term that mattered was tiny.

Perplexity is not the loss; it is two-to-the-loss. The loss is average surprise in bits; the perplexity, 2^{loss} , is the effective number of choices. A loss of 3 bits and a perplexity of 8 are the same statement.

The $\sqrt{d}$ is a temperature, not a typo. It holds the attention softmax at a constant warmth as dimension grows, undoing exactly the $\sqrt{d}$ spread that concentration of measure creates. Remove it and attention snaps brittle.

Essential Readings

THESE ARE REQUIRED. Each one makes precise something we built today by intuition, and each rewards a reading now far more than it would have a week ago, because you now know what to look for.

- **Vaswani et al., *Attention Is All You Need***

The paper that promoted attention from a patch to the whole architecture. Read Section 3.2 with this afternoon in mind: you will find our four fuzzy-dictionary exercises hiding inside the scaled dot-product equation, and the one sentence justifying the $\sqrt{d_k}$ — which you now know to read as a thermostat, not a stabiliser.

- **Devlin et al., *BERT: Pre-training of Deep Bidirectional Transformers***

The masked-word game, formalised. Section 3.1 is the rules of the card game in academic dress; pay attention to *why* bidirectionality (seeing both sides of the blank) is the whole claim, and recall how many bits it bought you in the game’s bidirectional round.

- **Reimers & Gurevych, *Sentence-BERT***

The cure for anisotropy, and the birth of the embedding models you will actually deploy. This is where the repulsive force becomes an architecture; read it as the bridge from “a model that predicts words” to “a model that places sentences for search.”

- **Radford et al., *Learning Transferable Visual Models from Natural Language (CLIP)***

The shared space that lets a paragraph fetch a photograph. Read it as the picture-book exercise’s foundation: one contrastive objective, two modalities, one globe.

- **Ethayarajh, *How Contextual Are Contextualized Word Representations?***

The careful, measured look at the anisotropic cone — the paper that quantifies the very histograms you will draw in the contrastive demonstration. It turns “the space is lopsided” from a claim into a measurement.

Oliver Twist’s List of Readings

OPTIONAL, BUT EACH ONE DEEPENS THE GRIN. These are for the students who went down the evening’s rabbit holes and came back wanting more.

- **Shannon, *A Mathematical Theory of Communication* (1948)**

The origin of $-\log p$ as a measure of information. Read the opening sections to watch surprise, entropy, and bits born in a single stroke — everything in Act I descends from this paper.

- **Jaynes, *Information Theory and Statistical Mechanics* (1957)**

The argument of Rabbit Hole 1, from the source. Jaynes shows that the Boltzmann distribution is not physics but *inference* — maximum entropy under a constraint — which is exactly why the softmax is everywhere.

- **Ramsauer et al., *Hopfield Networks Is All You Need* (2021)**

Rabbit Hole 3 in full. Dense, but the central result — that modern Hopfield retrieval *is* softmax attention — is one of the most quietly profound equivalences in the field.

- **Cover & Thomas, *Elements of Information Theory***

The textbook home for everything we touched in Act I: entropy, cross-entropy, KL divergence, and why minimising negative log-likelihood is minimising the distance between your model and the truth.

- **Su et al., *Whitening Sentence Representations* (2021)**

A different, linear-algebraic cure for anisotropy — worth reading next to Sentence-BERT to see that the same disease admits more than one treatment, and that the geometry is the thing, not any single method.

A Calibration Note

THIS SESSION WAS DENSE, and deliberately so. We covered an enormous distance: the softmax and its temperature, surprise and the negative log-likelihood, Emily's proof that the logarithm is forced rather than chosen, attention as a fuzzy dictionary lookup, the reason machines must speak machineese, self-attention disambiguating bank, the rung from token vectors to a pooled chunk embedding, the transformer as seq2seq with attention crowned, the masked-word game as masked language modelling, anisotropy and the repulsive cure of contrastive learning, search as motion on a globe, and a CLIP-powered picture-book over Qdrant — before the evening took the willing down four rabbit holes to find the same softmax hiding in coffee, in $\sqrt{d}$, and in a model of human memory.

If it feels like a great deal, notice that it was, in the end, *one* thing. We did not learn a dozen mechanisms today; we learned a single mechanism in a dozen disguises, and we learned to recognise it by its grin. The morning's stories — the rainy afternoon, the cow and the duck, Emily at the back of the room — were not scaffolding to be kicked away once the equations arrived. The equations *are* the stories, made exact. A model that learns is a model becoming less surprised; an embedding is a fossil of that unsurprise; attention is a librarian; and the whole of it is the Cheshire Cat, turning preferences into probabilities while assuming as little as it possibly can.

LET ME LEAVE YOU WITH A THOUGHT larger than any equation. What we built today is the machinery by which a machine decides what a word *means*, what a document is *about*, which photograph belongs beside which paragraph. These are not small powers. When we hand a system the authority to retrieve our laws, our medicine, our histories, and to decide which fragment of the human record surfaces in answer to a question, we are handing it a share of the custody of meaning itself. A softmax that quietly over-commits, an embedding space that quietly flattens a minority view into the majority cone — these are not merely technical defects; they are small distortions in the mirror a civilisation holds up to itself. This is why we refuse to treat any of this as a black box, why we insisted on building each piece by hand from weather and typos and coffee: a technology unmoored from the human understanding of *why* it does what it does is a technology waiting to be misused. You are learning the mathematics so that you remain its author, and never merely its operator.

We came in thinking the transformer was an equation to memorise. We leave knowing it is a sentence we can speak: a word, unsure of itself, asks its neighbours who it is, weighs their answers by how much they resonate, and becomes, for one sentence, certain. That is attention. Everything else is bookkeeping — and the bookkeeping is where the next three months live.

Next week we stop admiring the geometry and start cutting. Chunking — the seemingly humble question of *where to slice a document* — turns out to be the most consequential decision in a RAG pipeline, because every cut decides what that mean-pooled chunk vector will end up *meaning*. Bring today's globe with you. We are about to learn how to carve a library so that its pieces land in the right places on it.

References

- bahdanau2014attention Bahdanau, Dzmitry, Kyunghyun Cho, and Yoshua Bengio (2015). “Neural Machine Translation by Jointly Learning to Align and Translate”. In: **Proceedings of the 3rd International Conference on Learning Representations (ICLR)**. The original attention mechanism, introduced as soft alignment for sequence-to-sequence translation. arXiv: 1409.0473 [cs.CL].
- cover2006elements Cover, Thomas M. and Joy A. Thomas (2006). **Elements of Information Theory**. 2nd edition. Wiley-Interscience.
- devlin2018bert Devlin, Jacob et al. (2019). “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding”. In: *Proceedings of NAACL-HLT*.
- ethayarajh2019contextual Ethayarajh, Kawin (2019). “How Contextual Are Contextualized Word Representations? Comparing the Geometry of BERT, ELMo, and GPT-2 Embeddings”. In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, pages 55–65.
- jaynes1957statistical Jaynes, Edwin T. (1957). **Information Theory and Statistical Mechanics**. *Physical Review*, 106(4):620–630.
- radford2021clip Radford, Alec et al. (2021). “Learning Transferable Visual Models From Natural Language Supervision”. In: *Proceedings of the 38th International Conference on Machine Learning*.
- ramsauer2021hopfield Ramsauer, Hubert et al. (2021). “Hopfield Networks Is All You Need”. In: **International Conference on Learning Representations (ICLR)**.
- reimers2019sbert Reimers, Nils and Iryna Gurevych (2019). “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks”. In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, pages 3982–3992.
- shannon1948mathematical Shannon, Claude E. (1948). “A Mathematical Theory of Communication”. In: *The Bell System Technical Journal* 27.3, pages 379–423.

su2021whitening	Su, Jianlin et al. (2021). "Whitening Sentence Representations for Better Semantics and Faster Retrieval". In: arXiv preprint arXiv:2103.15316 .
taylor1953cloze	Taylor, Wilson L. (1953). "'Cloze Procedure': A New Tool for Measuring Readability". In: <i>Journalism Quarterly</i> 30.4, pages 415–433.
vaswani2017attention	Vaswani, Ashish et al. (2017). "Attention Is All You Need". In: <i>Advances in Neural Information Processing Systems</i> 30.

There were 39 pages in this document.

FEEDBACK

Please send any feedback on this document, or report errors to the author at asif@supportvectors.com

